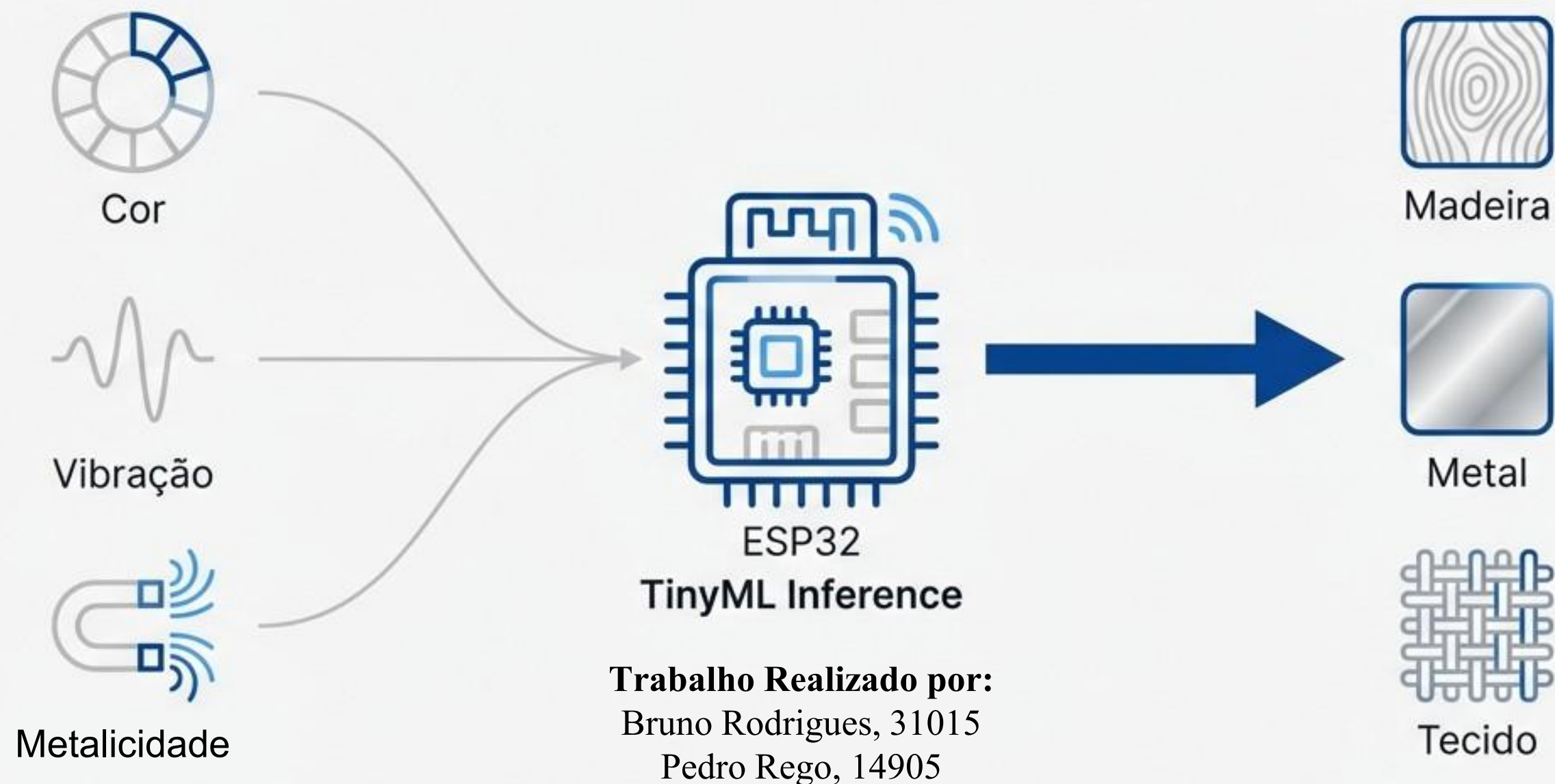


Classificação Robusta de Materiais na Indústria 4.0

Uma Arquitetura de Fusão Sensorial com Inferência TinyML em Edge Computing



Trabalho Realizado por:
Bruno Rodrigues, 31015
Pedro Rego, 14905

O Desafio: A Fragilidade dos Sistemas Unimodais em Ambientes Industriais

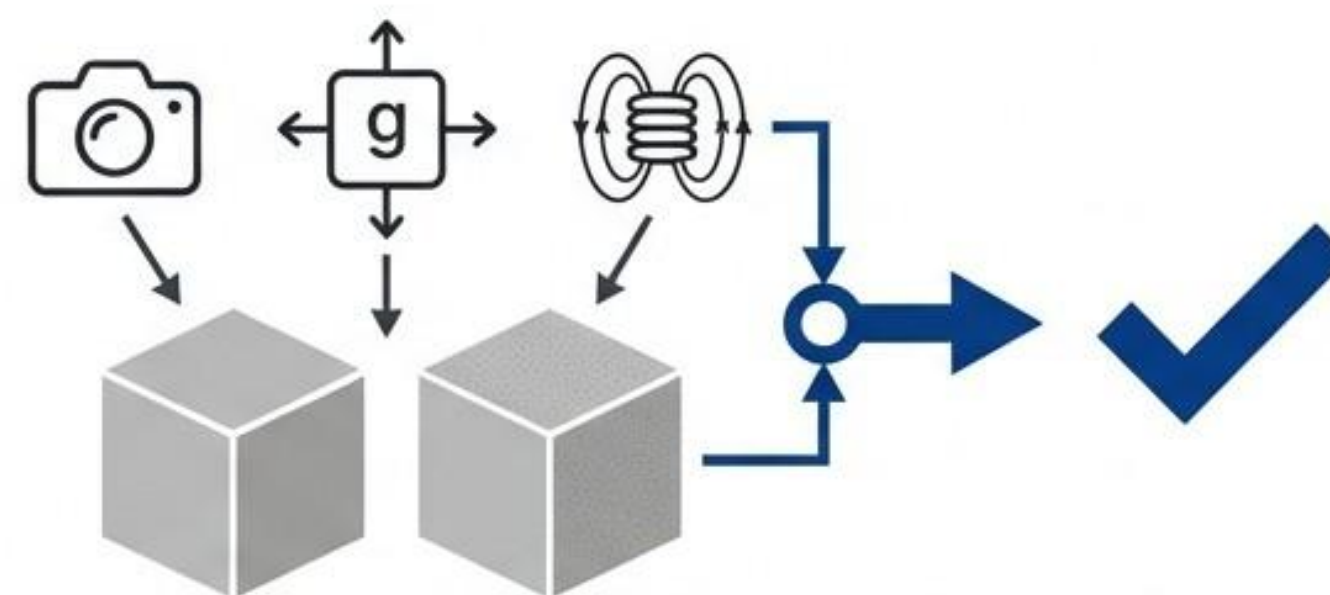
Soluções baseadas apenas em Visão por Computador são vulneráveis a variações de iluminação, sombreamento e reflectância similar entre materiais, resultando em classificações ambíguas e pouco fiáveis. O paradigma da Indústria 4.0 exige sistemas ciberfísicos (CPS) robustos e autónomos. A falha na classificação de materiais é um ponto crítico que limita a automação flexível.

A Nossa Abordagem Estratégica: Implementar uma arquitetura de **Fusão Sensorial**. Ao agregar dados de múltiplos sensores heterogêneos, reduzimos a incerteza e construímos um vetor de características com maior poder discriminatório, onde a fraqueza de um sensor é compensada pela força de outro.

Visão Unimodal



Fusão Sensorial



A Solução: Um Vetor de Características Ortogonal Processado no Edge

O sistema é construído em torno de um microcontrolador ESP32 que realiza a aquisição e o pré-processing de três domínios físicos distintos, garantindo uma representação rica e completa dos materiais.

Tabela 1 - Estrutura do Sistema

Variável Medida	Sensor Implementado	Domínio Físico	Justificação de Engenharia
Cor (Hue, Saturation, Value)	Sensor de Cor (Luz-Frequência)	Ótico	Essencial para distinguir materiais baseados na cor e não na intensidade.
Vibração (Vib)	Acelerómetro MEMS Triaxial (MMA8452)	Cinético/Tribológico	Permite a quantificação da rugosidade e textura da superfície através da análise do ruído de alta frequência.
Metallicidade (Metal)	Sensor Indutivo (ADC)	Eletromagnético	Fornece um forte sinal de presença/ausência de condutividade, crucial para a classe 'Metal'.

→ Cada sensor foi escolhido para capturar uma propriedade física única e complementar.

Firmware Inteligente: Garantindo Determinismo em Arquitetura Multicore

Contexto

O firmware, desenvolvido em C++, implementa uma Máquina de Estados Finitos (FSM) para gerir os modos de operação do sistema.

Estado 1: S_Idle (Monitorização e Calibração)

Funcionalidade: Web Server ativo para inspeção humana e calibração fina dos sensores via REST API.

Tecnologia: `ESPAsyncWebServer` para minimizar o bloqueio do core principal.

Estado 2: S_Acq (Aquisição de Alta Velocidade)

Funcionalidade: Streaming de dados de sensores via UART.

Decisão Crítica de Engenharia: Ao entrar neste estado, o Wi-Fi é completamente desativado (`WiFi.disconnect(true);`
`WiFi.mode(WIFI_OFF);`).

Justificação

As interrupções de rádio do Wi-Fi são assíncronas e introduzem latências estocásticas (variáveis) na leitura dos sensores. Desativar a pilha TCP/IP™ garante que o loop de aquisição opera com **timing quase determinístico**, assegurando a máxima fidelidade e consistência da amostragem.



ESP32 Data Fusion

Color Sensor (RGB)

R: 0 G: 0 B: 0
H: 0.00 S: 0.00 V: 0.00



MMA8452 Accelerometer



Vib: 1.22

Metal Sensor

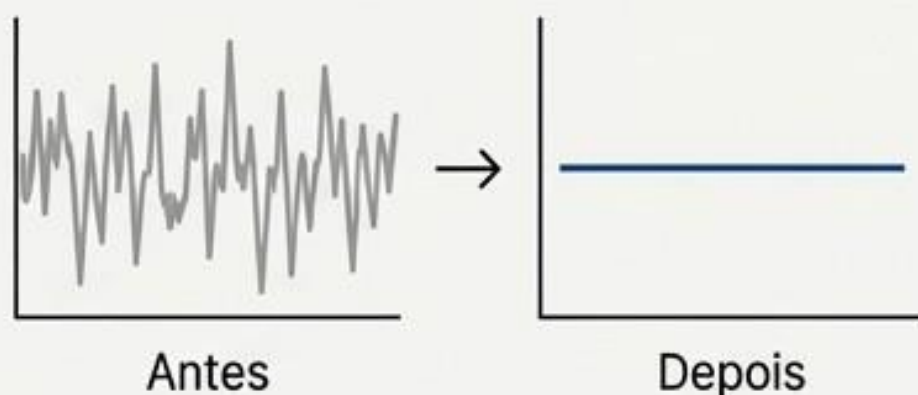


Metal: 0

Processamento de Sinal no Edge: Da Física Bruta a Características Robustas



Cor (Espectrofotometria)



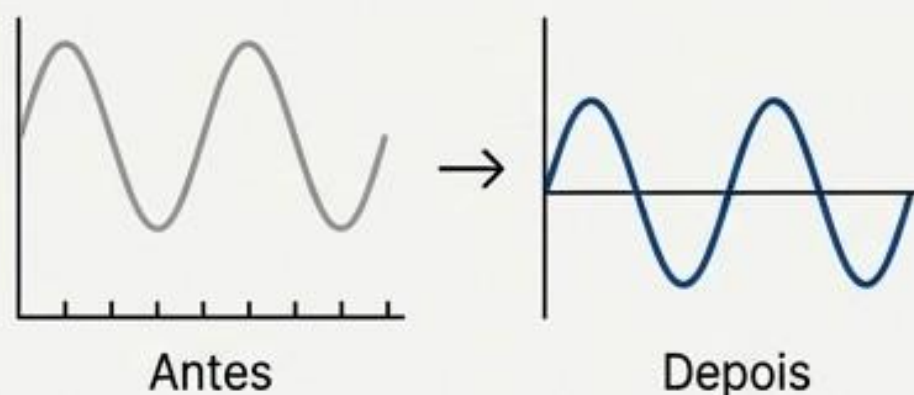
Desafio: A cor RGB é sensível a variações de iluminação.

Solução: Conversão para o espaço de cor **HSV**. O Matiz (Hue) e a Saturação (S) isolam a cromaticidade da luminância (Value), tornando a medição inerentemente robusta a flutuações de luz.

Técnica Adicional: Filtro de Média Móvel (N=8) para suavizar o sinal do sensor de



Vibração (Tribologia Dinâmica)

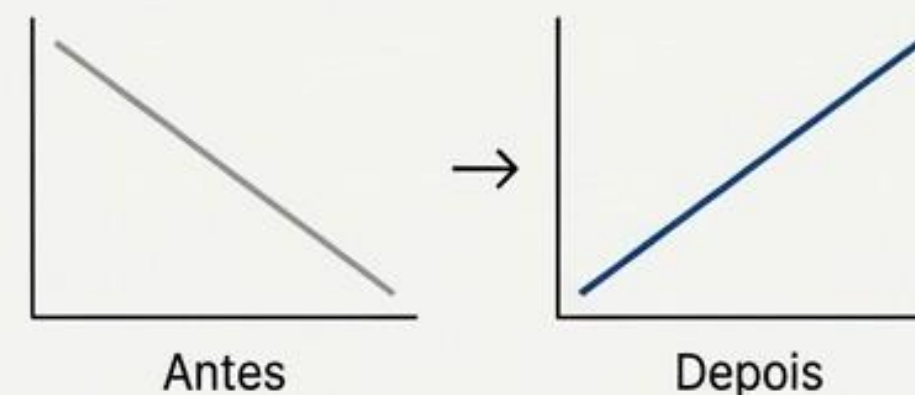


Desafio: O sinal do acelerômetro inclui a componente estática da gravidade ($\sim 9.8 \text{ m/s}^2$).

Solução: Aplicação de um **Filtro Passa-Alto (HPF) digital** de primeira ordem para remover a componente DC e isolar apenas as oscilações de alta frequência que representam a textura da superfície.



Metallicidade (Indução)



Desafio: O sinal bruto do sensor indutivo é inversamente proporcional à proximidade.

Solução: Inversão lógica e mapeamento do valor do ADC (12-bit) para garantir que a feature 'Metal' seja **monotonamente crescente** com a presença de metal, simplificando a aprendizagem do modelo.

A Ponte de Dados: Um Protocolo de Telemetria Robusto para um Dataset Íntegro

Arquitetura de Comunicação

Interface: UART a 115200 baud para maximizar o throughput sem erros de framing.

Modelo: Protocolo Comando-Resposta onde o host (Python) controla o início e o fim da aquisição.

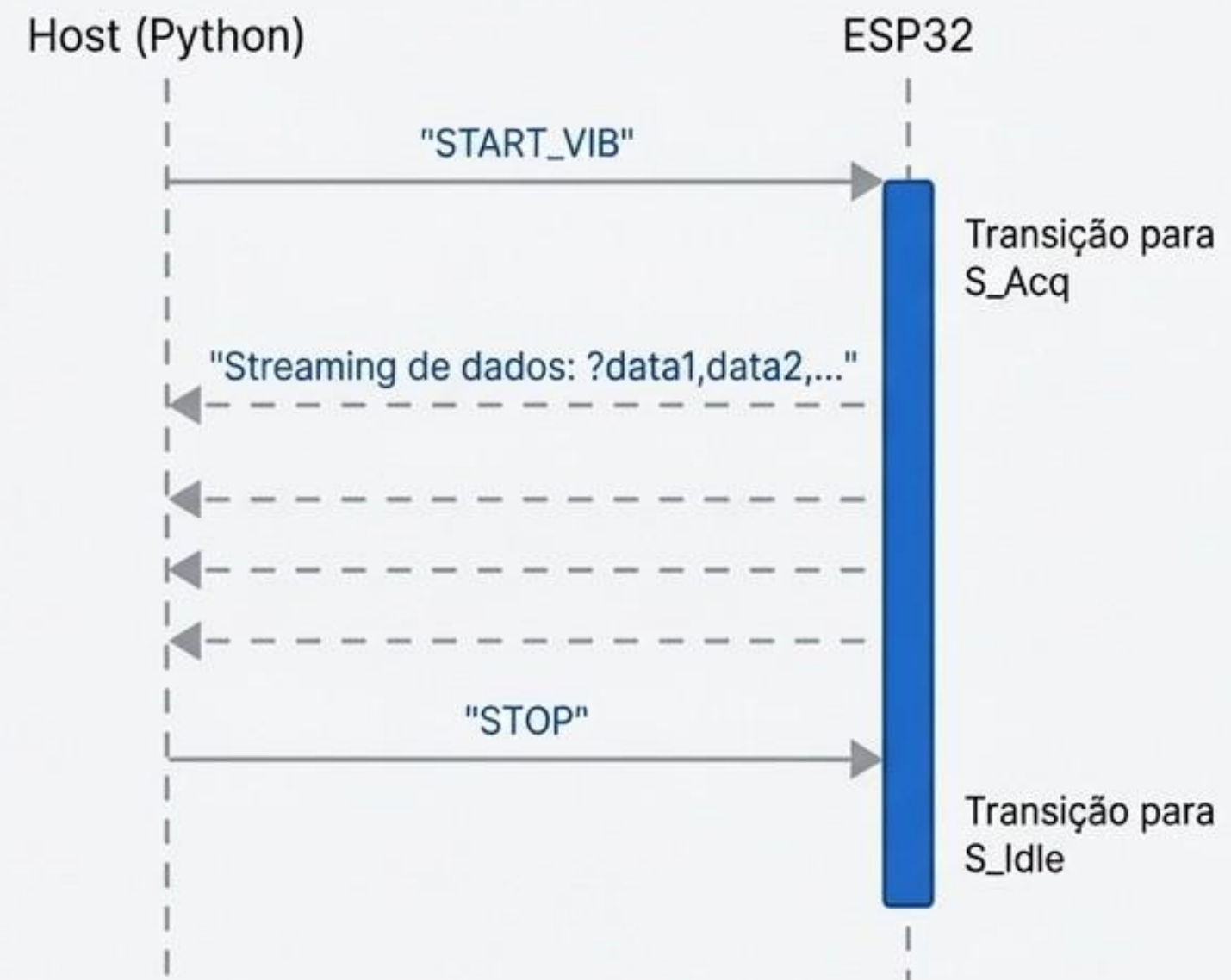
Mecanismo de Handshake e Serialização

1. **Iniciação (Host):** Envia um comando de controlo (e.g., ``START_HSV\n``).
2. **Resposta (ESP32):** Inicia o streaming de dados contínuo.
3. **Formato do Pacote:** Cada pacote de dados é prefixado com um delimitador claro (``?``), permitindo a fácil distinção entre dados e mensagens de debug (e.g., ``?25.5,100,50``).

Gestão no Host (Python)

Limpeza de Buffer: ``ser.reset_input_buffer()`` no início para eliminar dados residuais de sessões anteriores.

Decodificação Segura: ``decode('utf-8', errors='ignore')`` para prevenir que bytes malformados interrompam a aquisição.



O Desafio da Aquisição: Como Construir um Dataset Denso a Partir de Amostras Não-Simultâneas?

- A **Restrição Física**: As características exigem condições de medição distintas e mutuamente exclusivas:
 - **Cor (HSV)**: Requer estabilidade, iluminação constante e ausência de movimento.
 - **Vibração (Vib)**: Requer movimento deslizante e atrito contínuo contra a superfície.
- O **Problema**: É fisicamente **impossível** adquirir um vetor de características completo ([H, S, V, Vib, Metal]`) numa única instância de tempo.
- A Consequência: Uma aquisição ingênua resultaria num dataset **esparso**, com muitas células vazias, tornando-o **inútil** para o treino de modelos de Machine Learning.

Sample	H	S	V	Vibration	Metal
1	120	0.5	0.8	-	-
2	122	0.52	0.78	-	-
3	118	0.49	0.81	-	-
4	120	0.51	0.79	-	-
5	121	0.50	0.80	-	-
6	-	-	-	0.2	0.1
7	-	-	-	0.25	0.12
8	-	-	-	0.18	0.09
9	-	-	-	0.22	0.11

A Solução Algorítmica: Construção Incremental por Densificação Tabular

Estratégia: Um processo de duas fases gerido pelo script Python para transformar uma aquisição esparsa num dataset denso e coeso.

- **Fase 1: Inicialização e Esparsificação (Modo HSV)**

O script cria a estrutura do CSV e preenche as linhas com dados válidos de `CLASSE` e `Cor (H, S, V)`. As colunas `Vibration` e `Metal` são intencionalmente deixadas vazias ('').

- **Fase 2: Enriquecimento e Densificação (Modos VIB/METAL)**

O script adota uma estratégia de **Leitura-Modificação-Escrita**.

1. O conteúdo completo do CSV é lido para a memória.
2. O algoritmo localiza a primeira linha onde a coluna alvo está vazia.
3. Os novos dados de vibração são injetados sequencialmente a partir desse ponto.
4. O ficheiro CSV final e denso é **reescrito atomicamente no disco**.

Aquisição Esparsa

Sample	H	S	V	Vibration	Metal
1	120	0.5	0.8	-	-
2	120	0.5	0.8	-	-
3	120	0.5	0.8	-	-
4	120	0.5	0.8	-	-
5	-	-	-	0.2	0.1
6	-	-	-	0.2	0.1
7	-	-	-	0.2	0.1
8	-	-	-	0.2	0.1
9	-	-	-	0.2	0.1

Processamento em Memória



Dataset Denso Final

Sample	H	S	V	Vibration	Metal
1	120	0.5	0.8	[0.21]	0.1
2	120	0.5	0.8	[0.22]	0.1
3	120	0.5	0.8	[0.23]	0.2
4	120	0.5	0.8	[0.24]	0.2
5	120	0.5	0.8	[0.25]	0.1
6	120	0.5	0.8	[0.26]	0.2

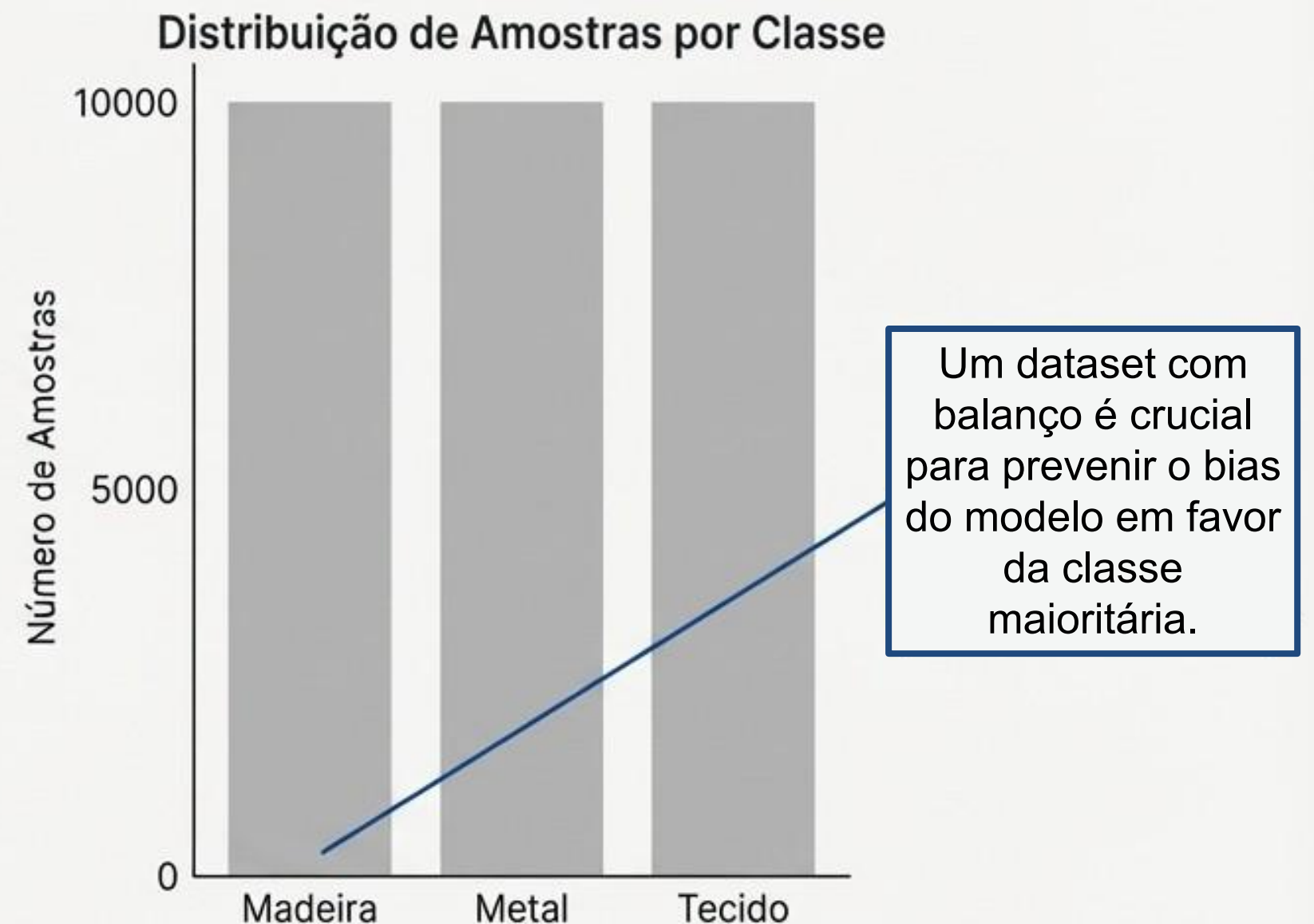
Escrita Atômica no Disco

A Fundação do Modelo: Um Dataset Exaustivo de 30.000 Amostras

Volume de Dados: Foram recolhidas **10.000 amostras válidas por cada uma das três classes** (Madeira, Metal, Tecido), resultando num dataset consolidado e balanceado de 30.000 observações.

Vantagens Estatísticas Críticas:

- **Mitigação de Overfitting:** A abundância de dados atua como um forte regularizador natural, permitindo o uso de redes neurais mais complexas sem risco de memorização.
- **Cobertura do Espaço de Características:** O modelo é exposto a uma maior diversidade de condições, garantindo que as fronteiras de decisão são mais robustas.
- **Redução da Variância de Erro:** As estatísticas do dataset (média, desvio padrão) são mais representativas da população real de medições.



Engenharia de Dados: A Análise Crítica de Variância e a Otimização do Vetor de Características

Contexto: Antes do treino, cada feature foi analisada quanto ao seu desvio padrão (σ). A normalização `StandardScaler` (Z-Score) falha se σ for próximo de zero, introduzindo valores `NaN` ou `Inf` que colapsam o treino do modelo.

A Descoberta: A análise de variância revelou que a feature `Metal` era constante ou tinha variância desprezível ($\sigma < 1e-6$) dentro de cada dataset individual antes da fusão.

Ação Corretiva: A feature `Metal` foi removida do conjunto de dados final, uma etapa essencial de limpeza de dados para garantir a estabilidade do treino.

Feature	Desvio Padrão (σ)	Estado	Ação Corretiva
Hue	Valor $\sigma > 1e-6$	Válida	Mantida
Saturation	Valor $\sigma > 1e-6$	Válida	Mantida
Value	Valor $\sigma > 1e-6$	Válida	Mantida
Vibration	Valor $\sigma > 1e-6$	Válida	Mantida
Metal	[0.0 ou valor $< 1e-6$]	Constante	REMOVIDA

Tabela 2 - Análise de Variância das Features e Ações Corretivas

Conclusão: O vetor de características final para o treino foi: **X_final = {Hue, Saturation, Value, Vibration}**.

A Competição de Modelos: Benchmarking para Identificar a Melhor Abordagem

Metodologia

Vários modelos de Machine Learning tradicionais (Scikit-Learn) foram treinados e avaliados para estabelecer uma linha de base de desempenho. Cada modelo foi executado 5 vezes para mitigar a variabilidade aleatória e calcular médias de desempenho.

Métricas de Avaliação

- **Accuracy:** Percentagem de previsões corretas.
- **F1-Score Ponderado:** A métrica superior, pois representa a média harmónica entre Precisão e Recall, sendo mais sensível a desequilíbrios no desempenho entre classes.

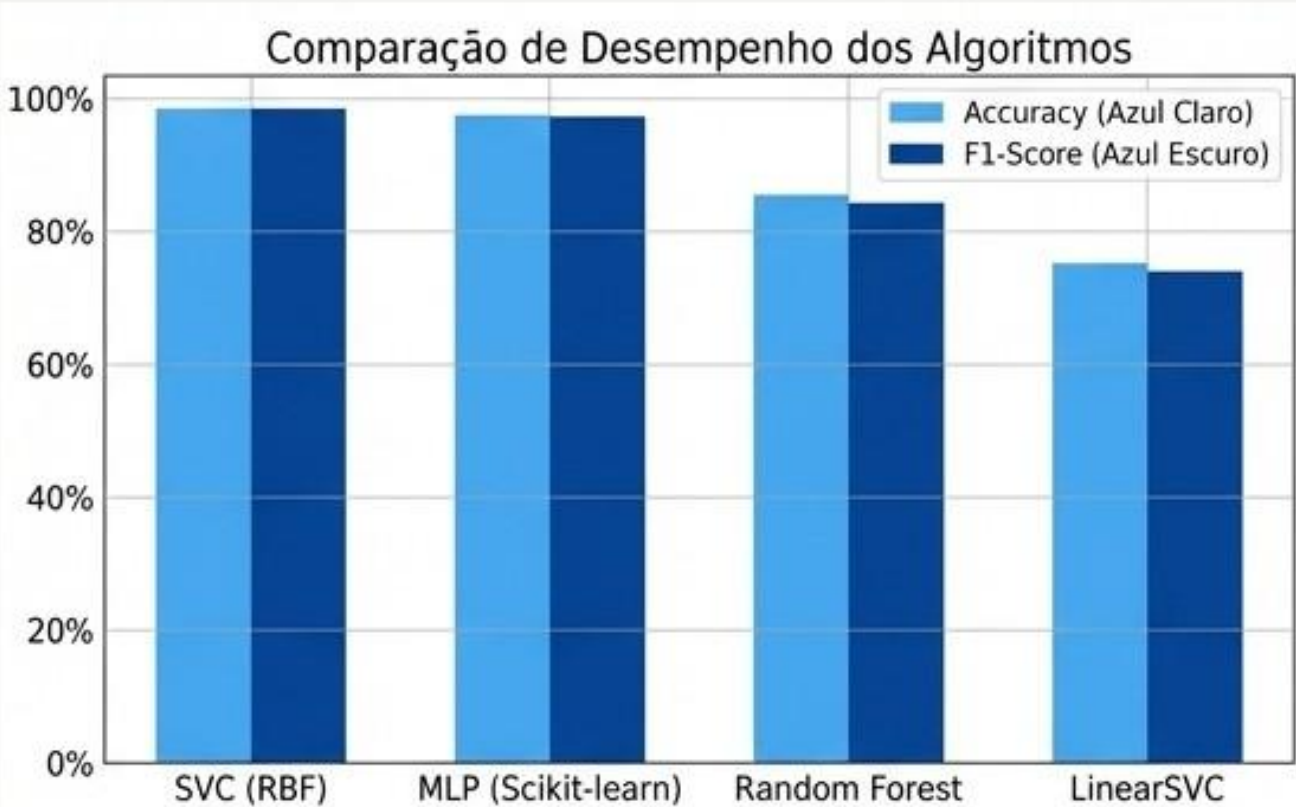


Tabela 3 - Resumo do Desempenho Médio

Algoritmo	Acc_Mean (%)	F1_Mean	Observação
SVC (RBF)	98.5%	0.985	Melhor desempenho bruto. Modela relações não-lineares.
MLP (Scikit-learn)	97.5%	0.975	Desempenho muito competitivo. Base para a rede final.
Outros (Ran.Forest, etc.)	85% - 75%	0.85 - 0.75	Menor capacidade de generalização para este problema.

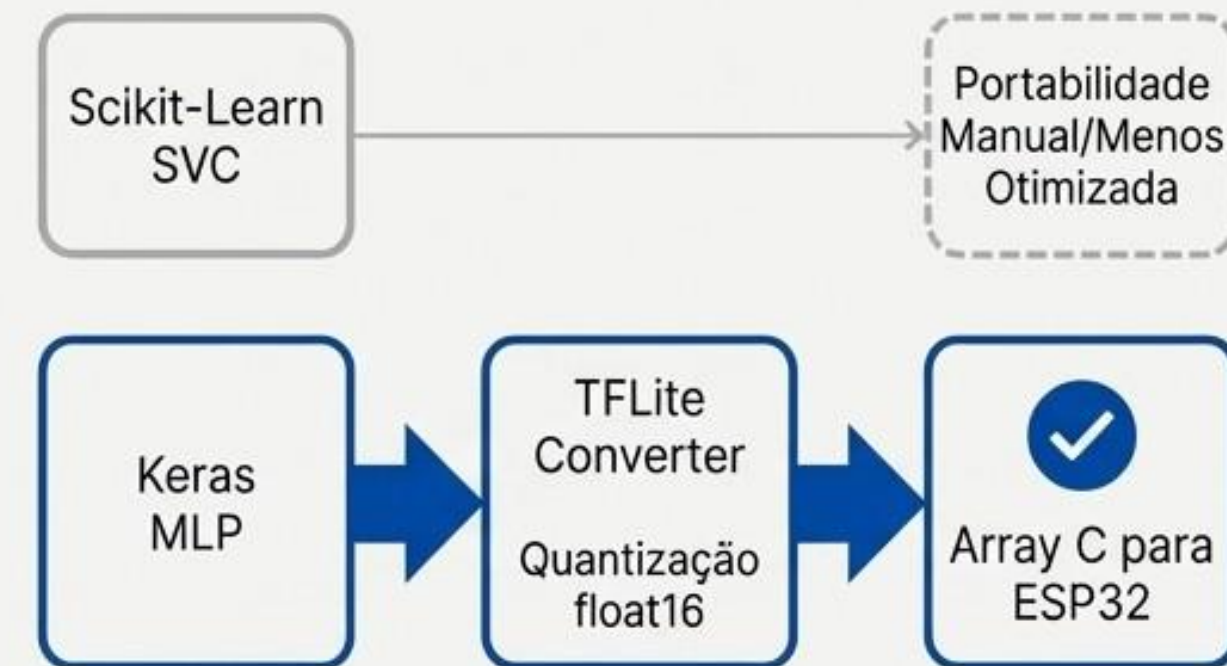
Conclusão Chave: Modelos capazes de aprender relações não-lineares (SVC com Kernel RBF e MLP) demonstraram uma superioridade clara.

A Decisão Estratégica: Porquê um MLP em Keras para a Solução Final?

Contexto: Apesar do excelente desempenho do SVC, o modelo final escolhido para implementação foi uma Rede Neuronal (MLP) desenvolvida em TensorFlow/Keras. A decisão não se baseou apenas na accuracy, mas na viabilidade e eficiência do sistema end-to-end.

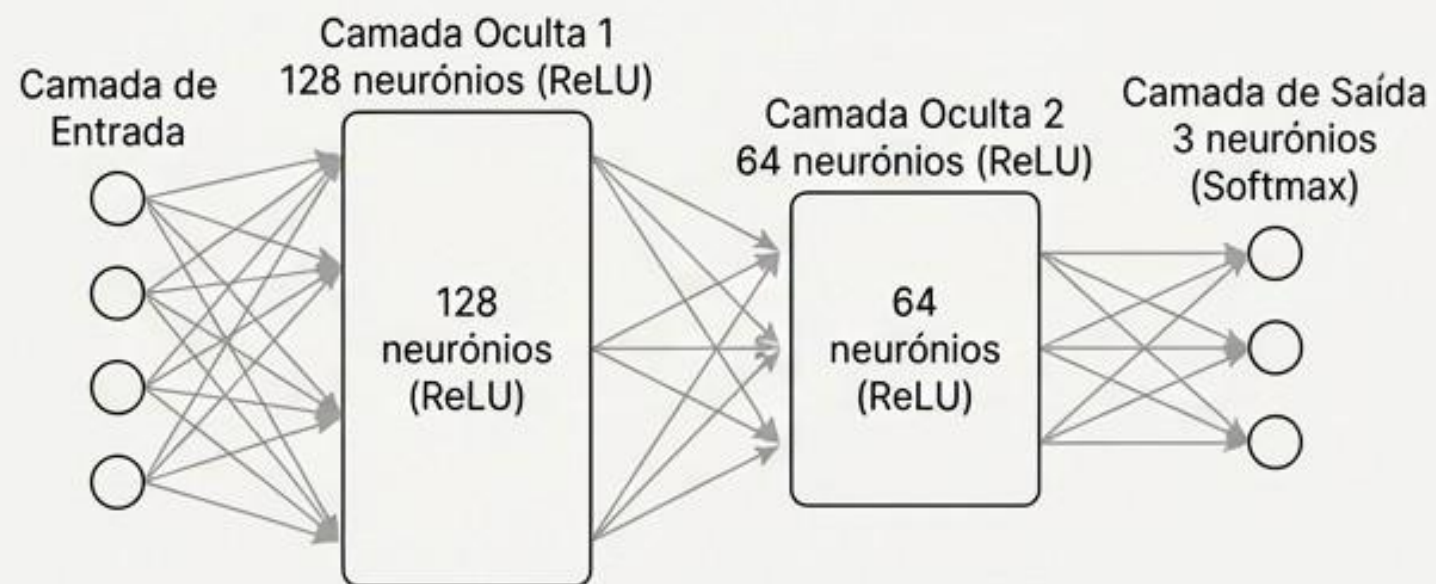
Justificação Técnica para a Escolha do MLP em Keras

- **Ecosistema TinyML Superior:** O pipeline de exportação `Keras -> TensorFlow Lite (TFLite) -> C-Array` é o padrão da indústria, sendo altamente otimizado e automatizado.
- **Otimização e Quantização Avançadas:** O TFLite permite a **quantização para float16**, que reduz o tamanho do modelo em aproximadamente 50% com perda de accuracy insignificante. Esta compressão é decisiva para a memória Flash limitada do ESP32.
- **Portabilidade e Facilidade de Implementação:** A conversão para um array C (model_data.h) é direta, enquanto a portabilidade de um modelo Scikit-learn para ambientes embarcados é frequentemente mais manual e menos otimizada.
- **Escalabilidade Futura:** A arquitetura MLP é inerentemente modular, permitindo a fácil adição de novas features ou classes no futuro.



O Modelo Final: Arquitetura Otimizada e Convergência Exemplar

Arquitetura da Rede Neuronal (MLP)

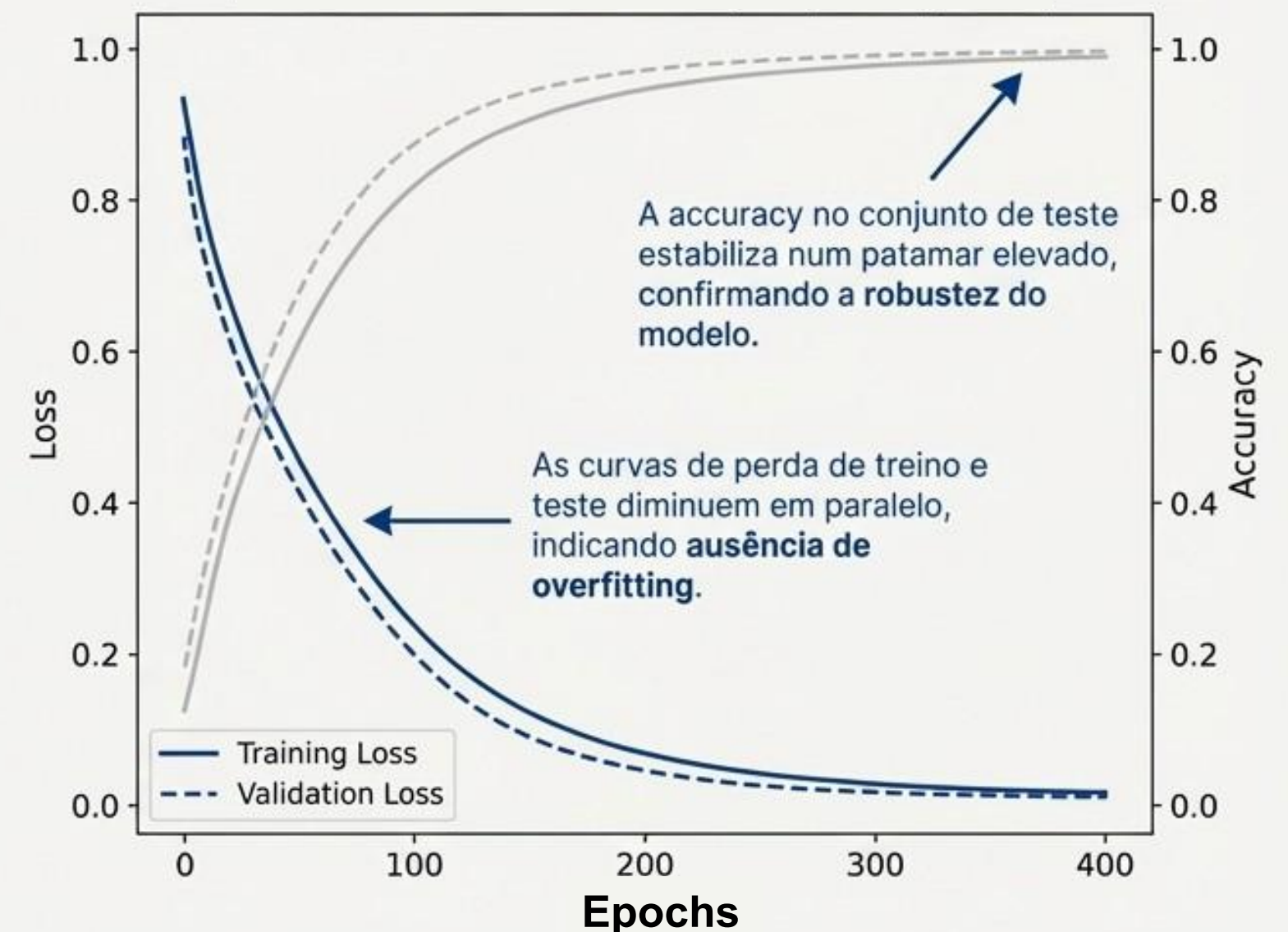


Estratégia de Treino

- **Otimizador:** Adam, robusto para datasets grandes.
- **Taxa de Aprendizagem:** Reduzida para $\eta = 0.0005$ para garantir uma convergência estável e precisa.
- **Epochs:** 400 epochs para assegurar a convergência total com uma taxa de aprendizagem baixa

Análise da Convergência

Curvas de Treino e Validação 400 epochs

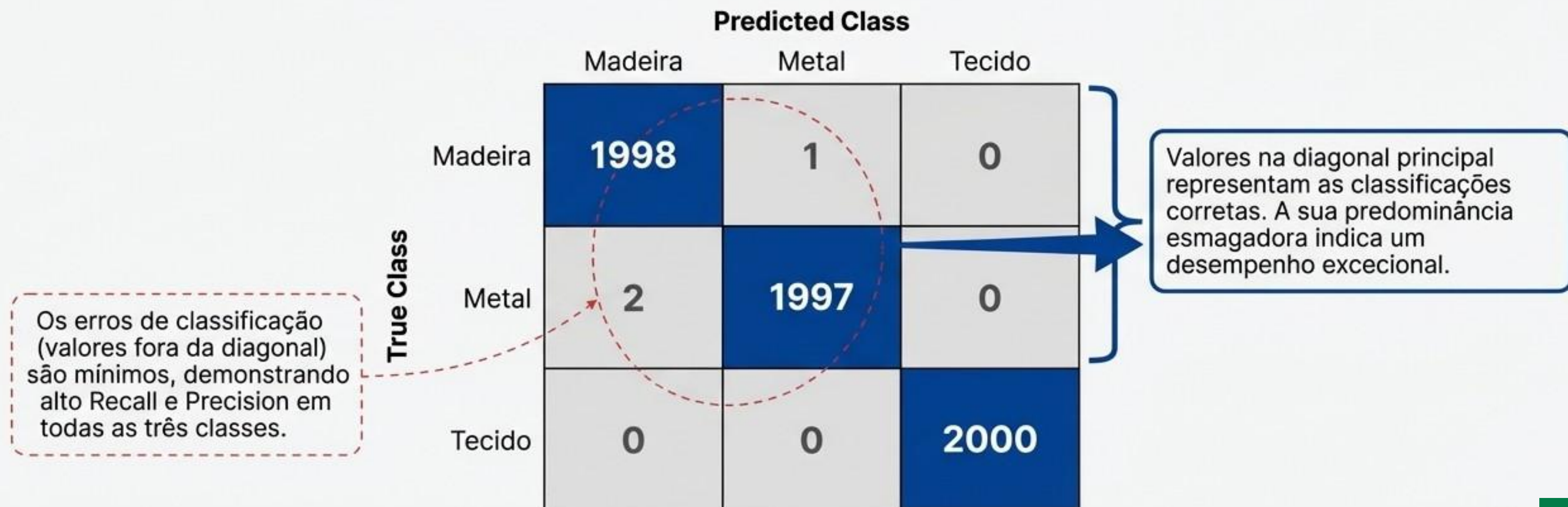


A Prova: Uma Precisão de 98.9% na Classificação de Materiais

Desempenho Final no Conjunto de Teste (6.000 amostras):

Accuracy: 98.9%

A matriz de confusão demonstra a distribuição dos resultados e a eficácia do modelo para cada classe individual. É o testemunho final do sucesso da abordagem de fusão sensorial.



Fechar o Ciclo: Da Prova Teórica à Solução Embarcada com TinyML

A fase final representa a ponte crítica entre o ambiente de treino Python e o ambiente de inferência com recursos limitados do ESP32.

